MAST32001 Computational Statistics - Lecture 13

Data Augmentation: Latent Variables and Predictor Augmentation

Klaus Nordhausen

University of Helsinki

WS 2026

Motivation

A unifying computational idea

A surprisingly useful strategy in computational statistics is:

> add variables that were not observed in the original data.

Why can adding variables help?

- a difficult likelihood can become conditionally simple;
- a difficult optimization can split into easy steps;
- simulation from a posterior can become possible by Gibbs sampling;
- artificial variables can expose otherwise hidden structural information.

Today we study two rather different uses:

1. **latent-data augmentation** for computation;
2. **predictor augmentation** for order determination.

In the **first part**, augmentation always means introducing **latent variables**: unobserved or missing quantities Z that are added to the representation of the model. Integrating them out returns the original observed-data model.

Observed data and augmented data

Let Y denote the observed data and Z an introduced latent variable.

The observed-data model is

$$p(y \mid \theta) = \int p(y, z \mid \theta)\, dz.$$

The integral may be difficult even when the complete-data model

$$p(y, z \mid \theta)$$

is simple.

The augmentation is useful when conditioning on Z creates a computationally convenient problem.

Three related views

The same augmented model can lead to different algorithms.

EM:

$$Z \mid Y, \theta^{(t)} \quad \Longrightarrow \quad E\{\log p(Y, Z \mid \theta) \mid Y, \theta^{(t)}\}.$$

Gibbs/data augmentation:

$$Z^{(t+1)} \sim p(z \mid Y, \theta^{(t)}), \qquad \theta^{(t+1)} \sim p(\theta \mid Y, Z^{(t+1)}).$$

Variational inference:

Approximate the latent-variable posterior by a tractable $q(Z, \theta)$.

Same modelling device; different computational strategy.

Connection to the ELBO from Lecture 12

For fixed θ, define

$$\mathcal{L}(q, \theta) = E_q\{\log p(y, Z \mid \theta)\} - E_q\{\log q(Z)\}.$$

Then

$$\log p(y \mid \theta) = \mathcal{L}(q, \theta) + \mathrm{KL}(q(Z) \,\|\, p(Z \mid y, \theta)) \,.$$

This gives a useful way to view EM:

- **E-step:** set $q(Z) = p(Z \mid y, \theta^{(t)})$, which makes the bound tight at the current $\theta^{(t)}$;
- **M-step:** maximize the bound with respect to θ.

So EM and variational inference are closely related coordinate-optimization ideas. The difference is that the EM E-step uses the **exact** conditional distribution of the latent variables, whereas VI usually restricts q to a tractable family.

Missing data and EM

A simple missing-data example

Suppose

$$Y_i \sim N(\mu, \sigma^2),$$

but some Y_i's are missing.

If the missing values are denoted by Z, then the complete data are

$$(Y_{\text{obs}}, Z).$$

For this simple independent-normal problem, EM is **not needed computationally**: the missing observations contain no additional information about μ and σ^2 beyond the observed values.

We use it only as a **conceptual illustration** of the augmentation idea:

- observed likelihood: integrate over missing values;
- augmented likelihood: imagine temporarily that the missing values were known;
- alternate between averaging over the missing information and updating parameters.

The Gaussian-mixture example below is a more substantive case where EM is genuinely useful.

The EM problem

Suppose the observed-data likelihood is

$$L(\theta; y) = p(y \mid \theta) = \int p(y, z \mid \theta)\, dz.$$

Direct maximization of

$$\ell(\theta; y) = \log p(y \mid \theta)$$

may be difficult because the latent variable Z has been integrated out.

But suppose the **complete-data log-likelihood**

$$\ell_c(\theta; y, z) = \log p(y, z \mid \theta)$$

would be easy to maximize if Z were known.

The EM algorithm exploits exactly this situation.

EM: the basic idea

Starting from a current parameter value $\theta^{(t)}$, EM alternates two steps.

E-step: compute

$$Q(\theta \mid \theta^{(t)}) = E_{\theta^{(t)}} \left[\ell_c(\theta; Y, Z) \mid Y = y \right].$$

M-step: update

$$\theta^{(t+1)} = \arg\max_{\theta} Q(\theta \mid \theta^{(t)}).$$

Then repeat until the parameter estimates or observed-data likelihood have stabilized.

EM stands for **Expectation–Maximization**.

What does the E-step do?

The complete-data log-likelihood depends on information Z that we do not observe.

At iteration t, we use the current parameter value to form

$$p(z \mid y, \theta^{(t)}).$$

We then average the complete-data log-likelihood over this conditional distribution:

$$Q(\theta \mid \theta^{(t)}) = \int \ell_c(\theta; y, z) p(z \mid y, \theta^{(t)}) \, dz.$$

For discrete latent variables, the integral becomes a sum.

So the E-step does **not** fill in one guessed value of the missing data. It averages over its current conditional uncertainty.

What does the M-step do?

After the E-step, $Q(\theta \mid \theta^{(t)})$ is an ordinary function of θ.

The M-step chooses

$$\theta^{(t+1)} = \arg\max_{\theta} Q(\theta \mid \theta^{(t)}).$$

The computational advantage is that maximizing Q can be much easier than maximizing the original observed-data likelihood.

Thus EM replaces one difficult optimization by a sequence of simpler optimization problems.

Why does EM make progress?

A fundamental property of EM is

$$\ell(\theta^{(t+1)}; y) \geq \ell(\theta^{(t)}; y).$$

So each complete E/M iteration does not decrease the observed-data log-likelihood.

This statement assumes the standard EM setting used here: the E-step is evaluated exactly and the M-step actually maximizes $Q(\theta \mid \theta^{(t)})$. Approximate variants such as Monte Carlo EM need not show perfectly monotone likelihood values at every iteration.

The underlying argument comes from the decomposition

$$\log p(y \mid \theta) = \log \frac{p(y, z \mid \theta)}{p(z \mid y, \theta)}$$

combined with conditional expectation and Jensen/KL arguments.

For this course, the important computational message is:

EM climbs the observed likelihood without maximizing it directly.

But monotonicity is not global optimality

EM's monotonicity does **not** imply that it finds the global maximum.

Possible issues:

- convergence to a local maximum;
- convergence to a saddle point or boundary solution;
- sensitivity to initialization;
- slow convergence near the optimum;
- different starts can produce different solutions.

Therefore, as with other iterative computational methods, initialization and convergence diagnostics matter.

A generic EM algorithm

1. Choose $\theta^{(0)}$.
2. For $t = 0, 1, 2, \ldots$:
   - **E:** compute $Q(\theta \mid \theta^{(t)})$;
   - **M:** maximize it to obtain $\theta^{(t+1)}$;
   - evaluate convergence.
3. Stop when, for example,

$$|\ell(\theta^{(t+1)}; y) - \ell(\theta^{(t)}; y)|$$

   or

$$\|\theta^{(t+1)} - \theta^{(t)}\|$$

   is sufficiently small.

In practice one should also inspect whether different initializations lead to the same solution.

Mixture models: allocations as missing data

For a two-component Gaussian mixture,

$$p(y_i \mid \theta) = \pi\phi(y_i; \mu_1, \sigma^2) + (1 - \pi)\phi(y_i; \mu_2, \sigma^2).$$

Introduce

$$Z_i \in \{1, 2\}.$$

Conditional on Z_i,

$$Y_i \mid Z_i = k \sim N(\mu_k, \sigma^2).$$

The awkward sum in the likelihood disappears from the complete-data likelihood.

This is the same latent-variable representation used in the course assignment.

EM for the mixture: E-step

For current parameters $\theta^{(t)}$, compute responsibilities

$$\tau_i^{(t)} = P(Z_i = 1 \mid Y_i = y_i, \theta^{(t)}) = \frac{\pi^{(t)} \phi(y_i; \mu_1^{(t)}, \sigma^{2(t)})}{\pi^{(t)} \phi(y_i; \mu_1^{(t)}, \sigma^{2(t)}) + (1 - \pi^{(t)}) \phi(y_i; \mu_2^{(t)}, \sigma^{2(t)})}.$$

The unobserved indicator $\mathbb{1}(Z_i = 1)$ is replaced by its conditional expectation.

The E-step therefore reconstructs the information that would have been available if the component labels were observed.

EM for the mixture: M-step

For a common variance, the updates include

$$\pi^{(t+1)} = \frac{1}{n} \sum_i \tau_i^{(t)},$$

$$\mu_1^{(t+1)} = \frac{\sum_i \tau_i^{(t)} y_i}{\sum_i \tau_i^{(t)}}, \qquad \mu_2^{(t+1)} = \frac{\sum_i (1 - \tau_i^{(t)}) y_i}{\sum_i (1 - \tau_i^{(t)})}.$$

The difficult mixture optimization has become a sequence of weighted standard problems.

EM for the mixture: common variance

If the common variance is **unknown**, it also has an M-step update:

$$\sigma^{2(t+1)} = \frac{1}{n}\sum_{i=1}^{n}\left[\tau_i^{(t)}(y_i - \mu_1^{(t+1)})^2 + (1-\tau_i^{(t)})(y_i - \mu_2^{(t+1)})^2\right].$$

Thus all parameters in the mixture model are updated by EM.

In **Interactive 1** we keep $\sigma^2 = 1$ known only to make the hand calculation short.

Interactive 1

Interactive task 1: one EM iteration

Consider three observations

$$y = (-1,\ 0,\ 2)$$

from a two-component Gaussian mixture with known variance $\sigma^2 = 1$.

At the current iteration,

$$\mu_1^{(t)} = -1, \qquad \mu_2^{(t)} = 1, \qquad \pi^{(t)} = 0.5.$$

1. Compute
$$\tau_i = P(Z_i = 1 \mid y_i, \theta^{(t)})$$
for all three observations.
2. Use these responsibilities to compute the updated
$$\pi^{(t+1)}, \quad \mu_1^{(t+1)}, \quad \mu_2^{(t+1)}.$$
3. Which observation has the most uncertain component membership?

Solution 1: E-step

For each observation,

$$\tau_i = \frac{0.5\,\phi(y_i; -1, 1)}{0.5\,\phi(y_i; -1, 1) + 0.5\,\phi(y_i; 1, 1)}.$$

```
y_em <- c(-1, 0, 2)
mu1 <- -1
mu2 <- 1
pi1 <- 0.5
tau <- pi1 * dnorm(y_em, mu1, 1) / (pi1 * dnorm(y_em, mu1, 1) +
     (1 - pi1) * dnorm(y_em, mu2, 1))
round(tau, 4)
```

```
## [1] 0.8808 0.5000 0.0180
```

Thus the observation at $y = 0$ has $\tau = 0.5$ and is the most uncertain: it lies exactly halfway between the two current component means.

The observations at -1 and 2 are much more strongly associated with components 1 and 2, respectively.

Solution 1: M-step

The updates are

$$\pi^{(t+1)} = \frac{1}{3}\sum_{i=1}^{3} \tau_i, \qquad \mu_1^{(t+1)} = \frac{\sum_i \tau_i y_i}{\sum_i \tau_i}, \qquad \mu_2^{(t+1)} = \frac{\sum_i (1-\tau_i) y_i}{\sum_i (1-\tau_i)}.$$

```
pi_new <- mean(tau)
mu1_new <- sum(tau * y_em) / sum(tau)
mu2_new <- sum((1 - tau) * y_em) / sum(1 - tau)
c(pi = pi_new, mu1 = mu1_new, mu2 = mu2_new)
```

```
##         pi        mu1        mu2
##  0.4662611 -0.6039711  1.1521393
```

Notice the structure:

- E-step: compute **soft component memberships**;
- M-step: compute weighted estimates as if those soft memberships were known.

One EM iteration maps

$$\theta^{(t)} \longrightarrow \theta^{(t+1)}.$$

EM versus Gibbs

With the same Z_i's:

EM

$$Z_i \rightsquigarrow E(Z_i \mid Y, \theta^{(t)}).$$

Gibbs

$$Z_i^{(t+1)} \sim p(Z_i \mid Y, \theta^{(t)}).$$

EM propagates a conditional expectation and seeks a mode/MLE.

Gibbs propagates random latent states and targets the full posterior.

This is an important recurring pattern in computational statistics.

Interactive 2

Interactive task 2: mixture augmentation

Consider

$$Y_i \mid Z_i = k \sim N(\mu_k, 1), \qquad P(Z_i = 1) = \pi.$$

Suppose

$$\mu_1 = 0, \qquad \mu_2 = 3, \qquad \pi = 0.4,$$

and we observe $y_i = 1$.

1. Derive $P(Z_i = 1 \mid Y_i = 1)$.
2. Compute its numerical value.
3. In one sentence, explain the difference between what EM and Gibbs do with this probability.

Solution 2: conditional allocation

Bayes' theorem gives

$$P(Z_i = 1 \mid y_i) = \frac{\pi \phi(y_i; \mu_1, 1)}{\pi \phi(y_i; \mu_1, 1) + (1 - \pi)\phi(y_i; \mu_2, 1)}.$$

At $y_i = 1$,

$$P(Z_i = 1 \mid y_i = 1) = \frac{0.4\phi(1; 0, 1)}{0.4\phi(1; 0, 1) + 0.6\phi(1; 3, 1)}.$$

```
num <- 0.4 * dnorm(1, 0, 1)
den <- num + 0.6 * dnorm(1, 3, 1)
num / den
```

```
## [1] 0.7492347
```

EM uses this probability as a fractional weight; Gibbs draws a Bernoulli allocation using this probability.

Probit regression by augmentation

Binary regression can hide a Gaussian model

In probit regression,

$$P(Y_i = 1 \mid x_i) = \Phi(x_i^\top \beta).$$

Introduce a latent Gaussian variable

$$Z_i = x_i^\top \beta + \varepsilon_i, \qquad \varepsilon_i \sim N(0,1),$$

and define

$$Y_i = \mathbb{1}(Z_i > 0).$$

Then

$$P(Y_i = 1 \mid x_i) = P(Z_i > 0 \mid x_i) = \Phi(x_i^\top \beta).$$

We have not changed the observed-data model; we have enlarged its representation.

Conditional distribution of the latent data

Given β and Y_i,

$$Z_i \mid Y_i = 1, \beta \sim N(x_i^\top \beta, 1) \text{ truncated to } (0, \infty),$$

and

$$Z_i \mid Y_i = 0, \beta \sim N(x_i^\top \beta, 1) \text{ truncated to } (-\infty, 0].$$

Conditional on Z, the model is simply Gaussian linear regression:

$$Z = X\beta + \varepsilon.$$

Thus augmentation converts a binary-response problem into alternating truncated-normal simulation and Gaussian regression.

Gaussian prior gives a Gaussian update

Let

$$\beta \sim N(b_0, B_0).$$

Given latent Z,

$$\beta \mid Z, Y \sim N(b_n, B_n),$$

where

$$B_n = (B_0^{-1} + X^\top X)^{-1}, \qquad b_n = B_n(B_0^{-1} b_0 + X^\top Z).$$

The non-Gaussian observed-data posterior has conditionally Gaussian structure after augmentation.

A probit Gibbs sampler

```r
rtrunc1 <- function(mu, y) {
  a <- if (y == 1) pnorm(0, mu, 1) else 0
  b <- if (y == 1) 1 else pnorm(0, mu, 1)
  qnorm(runif(1, a, b), mu, 1)
}
probit_da <- function(X, y, n_iter = 5000, prior_var = 100) {
  n <- nrow(X); p <- ncol(X)
  beta <- rep(0, p)
  out <- matrix(NA_real_, n_iter, p)
  B <- solve(crossprod(X) + diag(1 / prior_var, p))
  R <- chol(B)
  for (m in seq_len(n_iter)) {
    mu <- as.vector(X %*% beta)
    z <- vapply(seq_len(n), function(i) rtrunc1(mu[i], y[i]), numeric(1))
    b <- as.vector(B %*% crossprod(X, z))
    beta <- b + as.vector(t(R) %*% rnorm(p))
    out[m, ] <- beta
  }
  out
}
```

Interactive 3

Interactive task 3: probit latent variables

Suppose a single observation has

$$x_i^\top \beta = -0.5, \qquad Y_i = 1.$$

1. What is the conditional distribution of Z_i?
2. Is its conditional mean smaller or larger than -0.5?
3. Why would drawing $Z_i \sim N(-0.5, 1)$ without truncation be wrong?
4. What standard statistical problem do we obtain after all Z_i's have been sampled?

Solution 3

Because $Y_i = 1$, the latent variable must satisfy $Z_i > 0$. Hence

$$Z_i \mid Y_i = 1, \beta \sim N(-0.5, 1) \text{ truncated to } (0, \infty).$$

Its conditional mean is necessarily greater than 0, and therefore greater than the untruncated mean -0.5.

Sampling from the untruncated normal could produce $Z_i \leq 0$, contradicting the observed event $Y_i = 1$.

After all latent Z_i's are sampled,

$$Z = X\beta + \varepsilon$$

is an ordinary Gaussian regression problem.

Augmentation is not automatically efficient

Easier conditional steps can still mix slowly

A data-augmentation chain alternates

$$Z \mid \theta, Y \quad \text{and} \quad \theta \mid Z, Y.$$

Even if both draws are easy, Z and θ may be strongly dependent.

Then

$$\theta^{(t)} \to Z^{(t+1)} \to \theta^{(t+1)}$$

moves only a short distance.

Computational lesson:

simple conditional distributions do not guarantee an efficient Markov chain.

Centered and non-centered representations

A closely related computational issue is the **choice of parameterization**. This is not a new latent-data augmentation of the likelihood, but it illustrates the same principle: an equivalent representation can change computational efficiency dramatically.

A hierarchical model may be written as

$$\theta_i = \mu + \tau z_i, \qquad z_i \sim N(0,1).$$

Centered: work directly with θ_i.

Non-centered: work with z_i and reconstruct $\theta_i = \mu + \tau z_i$.

They define the same model but can have very different posterior geometry and MCMC efficiency.

Parameterization itself is therefore a computational choice.

What makes a useful augmentation?

A good augmentation balances several goals:

- conditional distributions should be easy;
- introduced variables should expose useful structure;
- dependence between augmented variables and parameters should not be excessive;
- the augmented representation should remain numerically stable;
- the cost per iteration should be reasonable.

There is rarely a universally optimal augmentation.

Predictor augmentation

A second, very different use of augmentation

So far we introduced **latent variables** whose integration recovers the original statistical model.

Predictor augmentation is conceptually different: we deliberately add variables that were **not** part of the original model.

Now consider the strategy

> add artificial noise variables to learn structural dimension.

Luo and Li (2021) call this **predictor augmentation**.

The goal is not to simplify a likelihood. It is to determine an unknown order/rank in dimension-reduction problems.

Order determination: what is the matrix M?

In many multivariate methods, the structural dimension can be expressed as the rank of a **method-specific population matrix** M:

$$\operatorname{rank}(M) = d.$$

The matrix is not the same in every method. For example:

- **PCA:** d is the number of covariance eigenvalues above an isotropic noise level; equivalently, the signal part of the covariance has rank d;
- **CCA:** d is the number of non-zero canonical correlations, equivalently the rank of an appropriate whitened cross-covariance matrix;
- **ICA:** d can represent the number of non-Gaussian signal components, using matrices that measure non-Gaussian structure;
- **sufficient dimension reduction:** matrices such as $\operatorname{Cov}\{E(X \mid Y)\}$ can have rank equal to the dimension of the relevant subspace.

So the common computational problem is: **estimate a rank/order d from noisy sample information.**

PCA intuition: signal plus isotropic noise

For a concrete picture, suppose the population covariance has a d-dimensional signal subspace and isotropic noise variance σ^2:

$$\Sigma = M_0 + \sigma^2 I_p, \qquad \operatorname{rank}(M_0) = d.$$

If V contains the population eigenvectors, then in the eigenvector coordinate system

$$V^\top \Sigma V = \begin{pmatrix} \Lambda_d & 0 \\ 0 & \sigma^2 I_{p-d} \end{pmatrix} = \begin{pmatrix} \Lambda_d - \sigma^2 I_d & 0 \\ 0 & 0 \end{pmatrix} + \sigma^2 I_p.$$

If d were known, the first d directions would be the signal subspace. The remaining directions form an isotropic noise subspace with no preferred orientation.

Why not just use a scree plot?

In the population idealization, the noise eigenvalues are all exactly σ^2. In a finite sample they are **not** exactly equal, and weak signal eigenvalues may blend into the noisy tail.

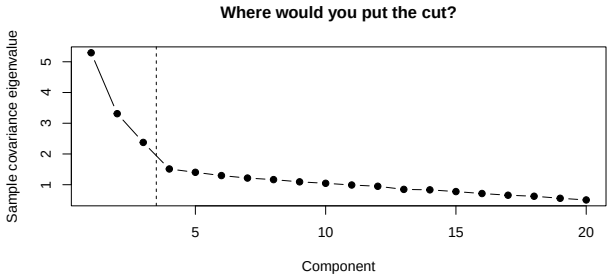
A scree plot therefore asks us to locate an elbow that may be visually ambiguous.

Also, a fixed rule based on "percentage of variance explained'' is only descriptive:

- it depends on the scaling of the original variables;
- a scientifically important direction can have small variance;
- with many noise variables, a large fraction of total variance may be noise.

Standardizing variables can address scale differences, but it does not solve the general order-determination problem.

A scree plot can have an unclear elbow



The dashed line shows the true $d = 3$, which would not necessarily be obvious from the scree plot alone.

Luo and Li: augment with artificial noise

Let $X \in \mathbb{R}^p$ be the original predictor and add an artificial noise vector

$$U \sim N(0, \sigma^2 I_r), \qquad U \perp X.$$

Form

$$X^* = \begin{pmatrix} X \\ U \end{pmatrix}.$$

For the PCA signal-plus-noise picture,

$$\Sigma^* = \begin{pmatrix} \Sigma & 0 \\ 0 & \sigma^2 I_r \end{pmatrix} = \underbrace{\begin{pmatrix} M_0 & 0 \\ 0 & 0 \end{pmatrix}}_{\text{signal part}} + \sigma^2 I_{p+r}.$$

The artificial coordinates are therefore **known to contain no signal**.

In practice, the noise level may be unknown

The augmentation distribution can itself contain nuisance parameters. In the PCA example,

$$U \sim N(0, \sigma^2 I_r),$$

so the noise level σ^2 must be specified.

- In an idealized derivation we can treat σ^2 as known.
- In real data it is often unknown and must be estimated from the data.
- The artificial variables should mimic the noise level reasonably well, so this estimation can matter in practice.

How nuisance parameters such as σ^2 should best be estimated is **beyond the scope of this lecture** and depends on the model and application.

For PCA, `PCAaug()` can either use a supplied value (`noise = "known"`, with `sigma2 = ...`) or estimate the noise variance using simple eigenvalue-based rules such as `"median"`, `"last"`, or `"quantile"`.

Why does this help?

If we knew the true signal subspace, a signal eigenvector of the augmented population matrix could be written

$$v_j^* = \begin{pmatrix} v_j \\ 0 \end{pmatrix}, \qquad j \le d,$$

so it has exactly zero mass in the artificial coordinates.

The noise eigenspace is different: it contains both the original noise directions and the newly added artificial directions. Because all these directions have the same population noise eigenvalue, there is no preferred orientation inside this enlarged noise subspace.

In sample data, the estimated noise eigenvectors can therefore **rotate and mix into the artificial coordinates**. This is the extra information created by augmentation.

What does “pointing into the artificial coordinates’ ’ mean?

Write a **unit-length** estimated augmented eigenvector as

$$\widehat{v}_j^* = \begin{pmatrix} \widehat{v}_{j,X} \\ \widehat{v}_{j,U} \end{pmatrix}, \qquad \|\widehat{v}_j^*\|^2 = 1.$$

Then

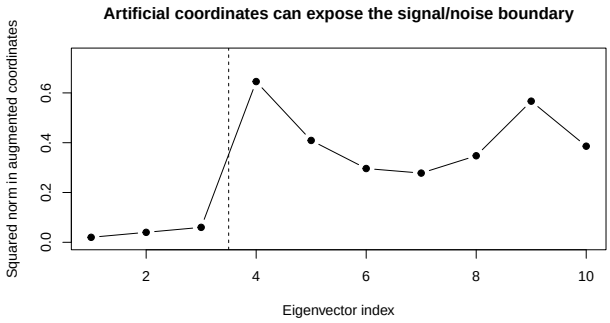
$$0 \le \|\widehat{v}_{j,U}\|^2 \le 1$$

is simply the fraction of its squared length carried by the artificial coordinates.

- near $\mathbf{0}$: the direction lies almost entirely in the original-variable coordinates;
- appreciably above $\mathbf{0}$: the direction has mixed with the known-noise coordinates.

We therefore expect small artificial-coordinate mass for the first d signal directions and a noticeable increase after entering the noise subspace.

Conceptual picture



The exact estimator is more carefully normalized, but this is the central diagnostic idea.

Combining two sources of information

Predictor augmentation uses:

1. **eigenvalue information**: in the dimension-reduction settings considered here, the eigenvalues provide one source of information about the signal/noise boundary;
2. **augmentation information**: signal eigenvectors tend to avoid artificial coordinates.

Luo and Li combine these to construct an order estimator.

The important computational-statistics principle is broader than the exact formula:

simulation of known-noise variables creates a reference against which structure can be detected.

Repeated augmentation

One artificial augmentation is random.

A practical implementation can repeat the augmentation:

$$U^{(1)}, \ldots, U^{(B)}$$

and average or otherwise summarize the augmentation contribution.

This reduces Monte Carlo variability introduced by the artificial predictors.

Now simulation is part of the estimator itself.

PCAaug() on simulated data

Here the true signal dimension is $d = 2$: two directions have variance larger than the common noise variance $\sigma^2 = 1$.

```
library(ICtest)

set.seed(1313)
n <- 1000
X_pca <- cbind(
  rnorm(n, sd = 2.5),
  rnorm(n, sd = 1.8),
  matrix(rnorm(n * 6), n, 6)
)
fit_aug <- PCAaug(X_pca,
                  noise = "median",
                  naug = 2,
                  nrep = 50)
```

PCAaug() on simulated data

```
summary(fit_aug)
```

```
##
## PCA Ladle estimator for X_pca
## k:  2
```

PCAaug() returns an object of class ladle. summary() is a convenient first look: it gives a compact summary including the estimated order k. Here we compare it with the known simulation truth $d = 2$.

Inspecting the fitted augmentation estimator

The fitted object also stores the estimated noise variance and the pieces of the criterion:

```
fit_aug$k        # estimated signal dimension
```

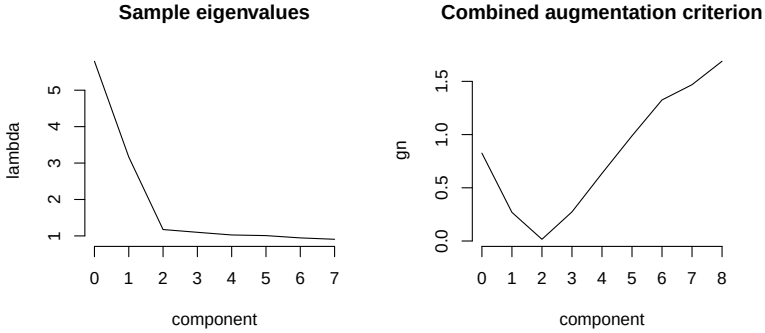
```
## [1] 2
```

```
fit_aug$sigma2  # noise variance used for augmentation
```

```
## [1] 1.062589
```

For programmatic work, `fit_aug$k` is the estimated order. The object also contains the covariance eigenvalues (`lambda`), the normalized eigenvalue contribution (`phin`), the augmentation/eigenvector contribution (`fn`), and their combined criterion (`gn`).

What does PCAaug() see?



The eigenvalues alone may suggest a cut, but `PCAaug()` combines them with the variation revealed by augmentation. The estimated order is the value where the combined criterion `gn` is minimized.

Other useful views are `ladleplot(fit_aug, crit = "fn")` for the augmentation/eigenvector contribution and `ladleplot(fit_aug, crit = "phin")` for the normalized eigenvalue contribution. Repeating the augmentation (`nrep > 1`) reduces Monte Carlo variability.

Interactive 4

Interactive task 4: order from augmented eigenvectors

Suppose the squared norms of the artificial parts of six ordered eigenvectors are

$$0.03, \quad 0.05, \quad 0.08, \quad 0.39, \quad 0.44, \quad 0.51.$$

The corresponding normalized eigenvalue contributions are

$$0.42, \quad 0.28, \quad 0.16, \quad 0.06, \quad 0.05, \quad 0.03.$$

1. What signal dimension would you suspect?
2. Which of the two sequences provides the clearer boundary here?
3. Why is the artificial-coordinate criterion informative?
4. Why might repeating the augmentation be preferable to using only one random U?

Solution 4

A natural suspected order is

$$\boxed{d = 3}.$$

The augmentation norms show a particularly sharp change between indices 3 and 4:

$$0.08 \longrightarrow 0.39.$$

The eigenvalues also decrease, but their transition is more gradual.

The artificial-coordinate criterion is informative because the added variables are generated independently and contain no genuine signal. Estimated signal directions should therefore have little reason to point into those coordinates, whereas noise directions can mix with them.

Repeating the augmentation reduces the Monte Carlo variability caused by one particular realization of the artificial noise.

Relation to other methods

Augmentation versus the ladle

Both address order determination, but use different extra information.

Ladle:

- eigenvalue information;
- bootstrap variability of estimated eigenspaces/eigenvectors.

Predictor augmentation:

- eigenvalue information;
- behavior of eigenvectors in artificially added noise coordinates.

Both illustrate a recurring computational idea:
create an auxiliary experiment that makes the signal/noise boundary more visible.

Augmentation versus knockoffs

Predictor augmentation adds artificial variables known to contain no signal.

Knockoffs also create artificial variables, but with a different goal:

- augmentation: help determine structural dimension/order;
- knockoffs: provide negative controls for variable selection while controlling false discoveries.

The next lecture develops this idea.

Practical cautions

What can go wrong?

Augmentation is not magic.

For latent-data augmentation:

- poor parameterization can cause slow MCMC;
- latent variables may be high-dimensional;
- easy conditional draws can still be expensive.

For predictor augmentation:

- finite-sample separation can be weak;
- results depend on how artificial predictors are generated and scaled;
- the number of added predictors is a tuning choice;
- repeated augmentation introduces computational cost.

Always diagnose the augmented procedure, not only its mathematical validity.

References

Main references for today's augmentation examples:

- Dempster, A. P., Laird, N. M. and Rubin, D. B. (1977): *Maximum likelihood from incomplete data via the EM algorithm*, **Journal of the Royal Statistical Society: Series B 39(1)**, 1–38. DOI: 10.1111/j.2517-6161.1977.tb01600.x.
- Albert, J. H. and Chib, S. (1993): *Bayesian analysis of binary and polychotomous response data*, **Journal of the American Statistical Association 88(422)**, 669–679. DOI: 10.1080/01621459.1993.10476321.
- Tanner, M. A. and Wong, W. H. (1987): *The calculation of posterior distributions by data augmentation*, **Journal of the American Statistical Association 82(398)**, 528–540. DOI: 10.1080/01621459.1987.10478458.
- Luo, W. and Li, B. (2021): *On order determination by predictor augmentation*, **Biometrika 108(3)**, 557–574. DOI: 10.1093/biomet/asaa077.
- Nordhausen, K., Oja, H., Perkonoja, K., Tyler, D. E. and Virta, J. (2026): *ICtest: Estimating and Testing the Number of Interesting Components in Linear Dimension Reduction*, R package version 0.3-7.

Summary

Take-home messages

1. In latent-data augmentation, adding **unobserved latent variables** can make a difficult problem computationally simple.
2. EM and Gibbs can use the **same augmented representation** in different ways.
3. Probit augmentation turns binary regression into truncated-normal sampling plus Gaussian regression.
4. Augmentation changes computational geometry; simple updates do not guarantee fast algorithms.
5. Luo–Li predictor augmentation adds **artificial noise predictors** to reveal structural order.
6. Simulation can therefore be part of the inferential procedure itself.
7. Artificial variables will appear again in knockoffs, but there they serve as negative controls for variable selection.